DATA ENGINEERING

Airflow Orchestration Visual Guide

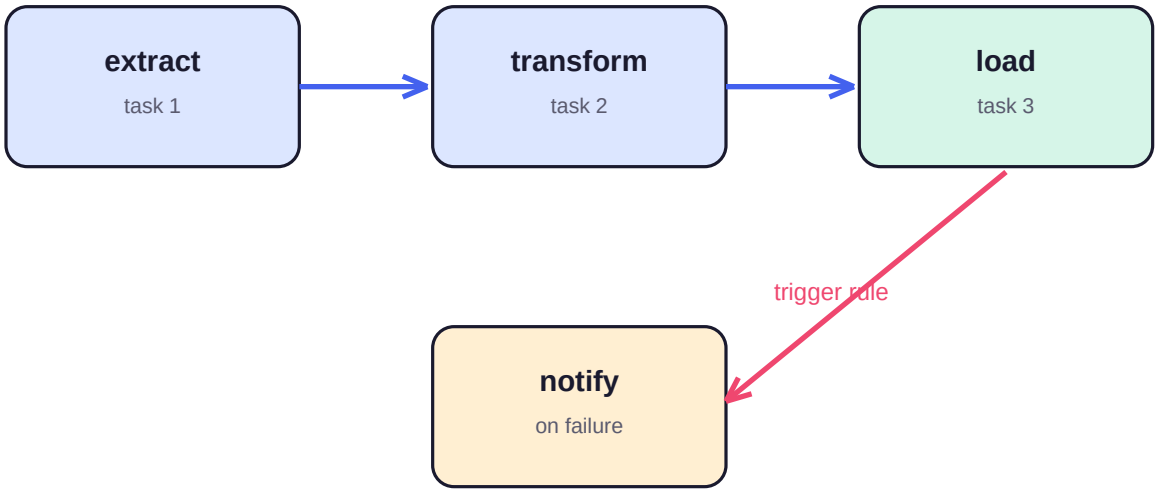
DAGs, operators, sensors, and backfills.
How pipelines get scheduled and supervised.

ANIMATED WALKTHROUGH
Step through pages to play the animation

STEP 1/6

DAG Anatomy

A Directed Acyclic Graph: tasks with dependencies, no cycles.



Edges are dependencies: transform waits for extract. Trigger rules (all_success, one_failed…) control when a task fires.

STEP 2/6

Operators: The Building Blocks

Each task is an operator — a template for a unit of work.

BashOperator

Run a shell command

PythonOperator

Call a Python function

SQLOperator

Execute SQL in a database

Sensor

Wait for a condition (file, partition)

Deferrable operators (async) free the worker slot while waiting — critical at scale. Prefer them over polling sensors.

- Providers add 100+ operators: S3, Snowflake, dbt, Kubernetes…
- TaskFlow API (@task) wires XComs with plain Python returns
- Keep tasks idempotent — Airflow WILL retry them

STEP 3/6

Sensors: Wait Smartly

Sensors pause a DAG until the world is ready.

FileSensor
wait for landing file

ExternalTaskSensor
wait for upstream DAG

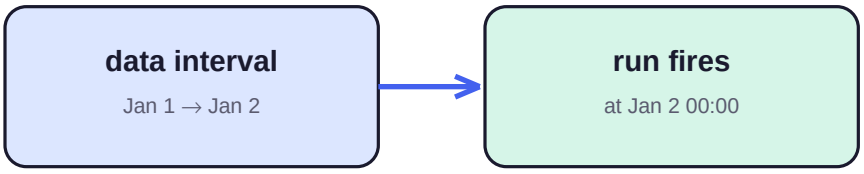
poke_interval: check every N seconds.
mode='reschedule': release the worker slot between pokes —
a poking sensor should never hold a slot hostage.

Reschedule mode (or deferrable sensors) is the difference between a healthy cluster and one deadlocked by waiting tasks.

STEP 4/6

Scheduling & Backfill

Airflow schedules at the END of each interval — and can replay history.



A @daily DAG's Jan-1 run starts when Jan 1's data is complete —
at the START of Jan 2. This confuses everyone exactly once.

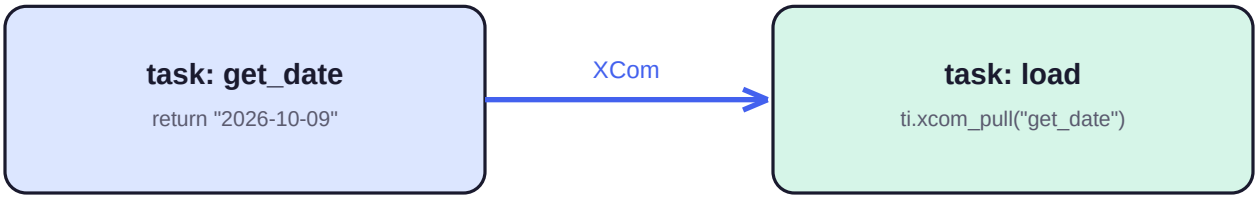
Backfill: airflow dags backfill --start-date 2026-01-01 replays every missed interval — how you rebuild history after a logic fix.

- catchup=True (default): missed intervals queue up on deploy
- Set catchup=False for DAGs that should only run forward
- Timetables: cron, deltas, or custom dataset-driven schedules

STEP 5/6

XComs: Passing Small Data

Cross-communication: tasks share small values via the metadata DB.



XComs are for small values (IDs, dates, row counts) — NOT dataframes. Push big data to storage and pass the path.

- TaskFlow: `return` auto-pushes; function args auto-pull
- Custom XCom backends (S3/GCS) lift the size limit
- Never pass credentials via XCom — use Connections

STEP 6/6

Task Lifecycle & Retries

Every task moves through states — design for the red ones.

queued → running → success

failed → up_for_retry → running

upstream_failed / skipped

retries + retry_delay + exponential backoff turn transient failures (API hiccups) into non-events. Alert only on final failure.

- SLAs: alert when a task runs longer than expected
- Pools: cap concurrency for fragile external systems
- on_failure_callback: page the human, with context

STEP ✓/✓

Key Takeaways

Airflow Orchestration — Remember

- DAGs declare task dependencies; the scheduler executes them in topological order.
- Operators are task templates; sensors wait for conditions — use reschedule/deferrable modes.
- Runs fire at the END of each data interval; backfill replays history after logic fixes.
- XComs pass small values between tasks; big data goes through storage, not the metadata DB.
- Make tasks idempotent and retryable — retries with backoff absorb transient failures.

Keep exploring

Pair this guide with the matching video walkthroughs and hands-on labs at interstitiumlabs.dev